



Education

Lab 4.1. Installing Argo Workflows

In the previous chapter, we discussed the fundamentals of Argo Workflow, exploring its capabilities. A workflow engine designed for Kubernetes, enabling the orchestration of complex tasks and processes within containerized environments. It provides a powerful and flexible framework for defining, running, and managing workflows as a series of interconnected steps. In this chapter, we will focus on installation, accessing the graphical user interface, installing CLI, and finally exploring a few of Argo Workflow's native Kubernetes resources.

Objective

Install Argo Workflows and understand how to access Argo Workflows resources and Argo UI.

Prerequisites

- Basic understanding of Docker, Kubernetes, and command-line interface operations.
- Access to a computer with an internet connection.
- An installation of Kubernetes that you have full control over
 - See Chapter 2's "Deploying Kubernetes for Argo" section for details on how to set one up for yourself

Install Cluster and Argo Workflows

1. Deploy Argo Workflows

Argo Workflows is a separate project under the Argo family of tools. In previous labs, you may have installed Argo CD in its own namespace. While you can install all of the Argo tools under a common Kubernetes namespace, it is best to keep each tool like Workflows, Rollouts, and ArgoCD in their own namespaces. They work very well together, but can work separately.

Create a namespace for Argo Workflows with the following command:

```
kubectl create namespace argo
```

Deploy Argo Workflows using the quick start manifest. Run the following command:

```
kubectl apply -n argo -f
https://github.com/argoproj/argo-workflows/releases/download/v3.7.3/install.yaml
```

Verify that Argo Workflows is installed by running the following command:

```
$ kubectl -n argo get pods
```

Output:

NAME	READY	STATUS	RESTARTS
AGE			
argo-server-5fdcf6fd6-tfx6m	1/1	Running	0
2m			
workflow-controller-5df59665c9-zz92b	1/1	Running	0
2m			

2. Accessing UI Dashboard

Patch argo-server authentication

The Argo-server (and thus the UI) defaults to client authentication, which requires clients to provide their Kubernetes bearer token to authenticate. For more information, refer to the [Argo Server Auth Mode documentation](#). We will switch the authentication mode to the server so that we can bypass the UI login for now. Run the following command:

```
kubectl patch deployment \
  argo-server \
  -n argo \
  --type='json' \
  -p='[{"op": "replace", "path":
"/spec/template/spec/containers/0/args", "value": [
  "server",
  "--auth-mode=server"
]}]'
```

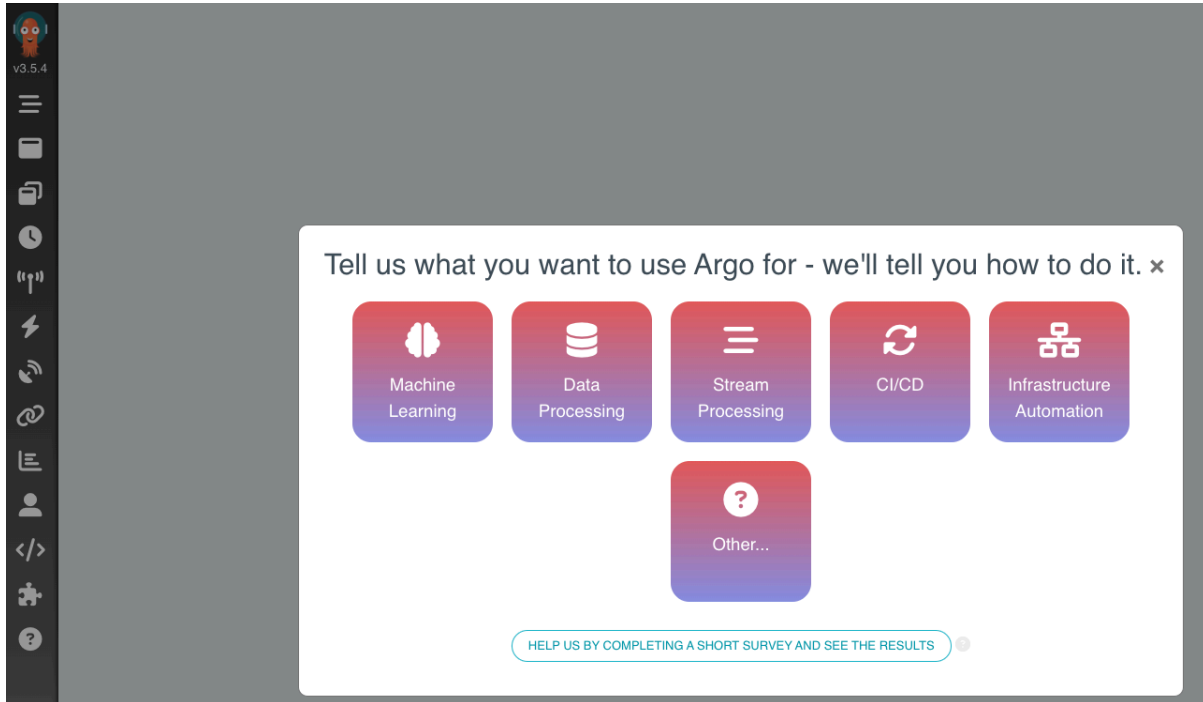
Port-forward the UI

Now you can use port-forward, so you can access the UI. Run the following command:

```
kubectl -n argo port-forward deployment/argo-server 2746:2746
--address 0.0.0.0
```

The Argo Workflows UI will be accessible at <https://localhost:2746>. Since we are using a self-signed certificate, you need to bypass a TLS error.

Please use **HTTPS** instead of **HTTP** protocol. while navigating to Argo Workflow UI, otherwise it will cause a server-side error that will break the port forwarding.



Start Screen of Argo Workflows

3. Install the Argo Workflows CLI

Download the latest Argo Workflows version from [releases](#). More detailed installation instructions can be found via the [CLI installation documentation](#).

On Ubuntu 24.04, you can install the CLI with the following commands:

```
$ wget
https://github.com/argoproj/argo-workflows/releases/download/v3.7.3/argo-linux-amd64.gz -O argo.gz

$ gunzip "argo.gz"
```

```
$ chmod +x argo
```

```
$ sudo mv argo /usr/local/bin/
```

After doing so, verify that the `argo` CLI is installed correctly by running the following command:

```
$ argo version
```

Output:

```
argo: v3.7.3
  BuildDate: 2025-10-14T11:39:00Z
  GitCommit: bded09fe4abd37cb98d7fc81b4c14a6f5034e9ab
  GitTreeState: clean
  GitTag: v3.7.3
  GoVersion: go1.24.7
  Compiler: gc
  Platform: linux/amd64
```

4. Optional: Enable Shell auto-completion

To get easy access to Argo Rollout Resources, the CLI can add shell completion code for several shells. For bash, you can use the following command:

```
source <(argo completion bash)
```

Other shells are also supported. Please refer to the [completion command documentation](#) for more details.

5. Using Argo Workflows

The `argo` CLI tool is primarily for the Argo Workflows management via the CLI. It works similarly to the Argo CD CLI in that it can either use the Kubernetes API or work directly with an Argo Server.

This lab will primarily use the web GUI.

More details are listed in the `argo help` output:

```
$ argo help
```

Output:

```
You can use the CLI in the following modes:
```

Kubernetes API Mode (default)

Requests are sent directly to the Kubernetes API. No Argo Server is needed. Large workflows and the workflow archive are not supported.

Use when you have direct access to the Kubernetes API, and don't need large workflow or workflow archive support.

Here is a list of the most common commands to operate with Argo Workflows:

```
argo list # list workflows
argo get <workflow-name> # display details about a workflow
argo submit <workflow-file.yaml> # submit a workflow
argo watch <workflow-name> # watch workflow progress
argo template list # list workflow templates
argo logs <workflow-name> # display logs for workflow pod(s)
```